

Quicksort tracing

15 5 9 2 1 8
 $\uparrow$ $\uparrow$
 $l=0$ $r=5$

→ Quicksort (A[0..5])
if $l < r$

$\bar{q} \leftarrow \text{partition}(A[l..r])$

→ Partition(A[0..5])

$p \leftarrow A[l] = 15$

15 5 9 2 1 8
 $\uparrow$ $\uparrow$
 i, l $j = r+1$

15 5 9 2 1 8
 $\uparrow$ $\uparrow$
 i, j

Swap(A[l], A[j])

8 5 9 2 1 15
 $\boxed{p \leftarrow 5 \text{ is returned}}$
→ partition(A[0..5])
0..5-1

8 5 9 2 1
 $\uparrow$ $\uparrow$ $\uparrow$
 i, l $p=8$ $r+1$

8 5 9 2 1
 $\uparrow$ $\uparrow$

Swap($A[i]$, $A[j]$)

8 5 1 2 9
 $\uparrow_i$ $\uparrow_j$ $\uparrow_k$

Swap($A[k]$, $A[j]$)

2 5 1 8 9
 $\uparrow_j$ $\uparrow_k$

$k \leftarrow j$ is returned

Partition($A[0 \dots 2]$)
0 $\dots$ 8-1

2 5 1
 $\uparrow_{i, l}$ $\uparrow_{j=r+1}$
 $p=2$

2 5 1
 $\uparrow_i$ $\uparrow_j$

2 1 5
 $\uparrow_l$ $\uparrow_j$ $\uparrow_k$

Swap($A[j]$, $A[k]$)

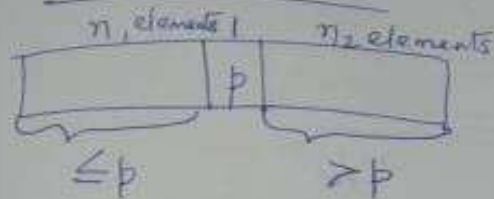
1 2 5

$k \leftarrow 1$ is returned

Thus, the whole array is sorted.

1 2 5 8 9 15

Time complexity of Quicksort



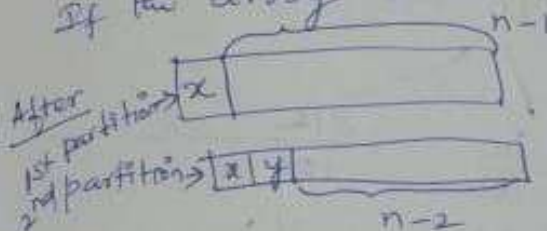
$$n_1 + n_2 + 1 = n$$

$$T(n) = T(n_1) + T(n_2) + cn$$

$\rightarrow cn$ is the time for partitioning the array.

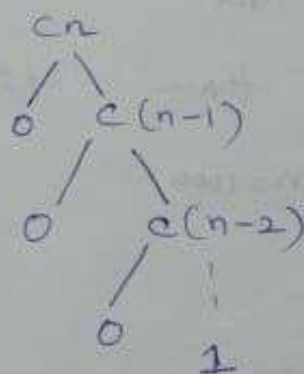
Worst case

If the array is sorted



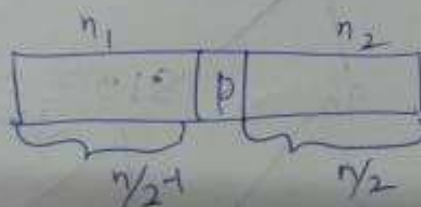
and so on.

$$T(n) = T(n-1) + cn$$



$$\begin{aligned}
 &= c[n + (n-1) + (n-2) + \dots + 1] \\
 &= c \left[\frac{n(n+1)}{2} \right] \\
 &= O(n^2)
 \end{aligned}$$

Best case



$$n_1 \approx \frac{n}{2} \approx n_2$$

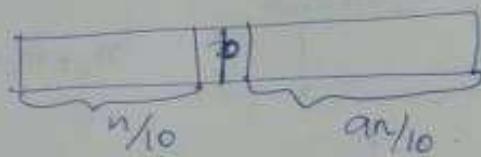
$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + cn$$

$$= 2T\left(\frac{n}{2}\right) + cn$$

Applying master's theorem we get

$$T(n) = O(n \log n)$$

Almost best case



$$T(n) = T\left(\frac{n}{10}\right) + T\left(\frac{9n}{10}\right) + cn$$

level No of nodes

0 1

1 2

2 4

3 8

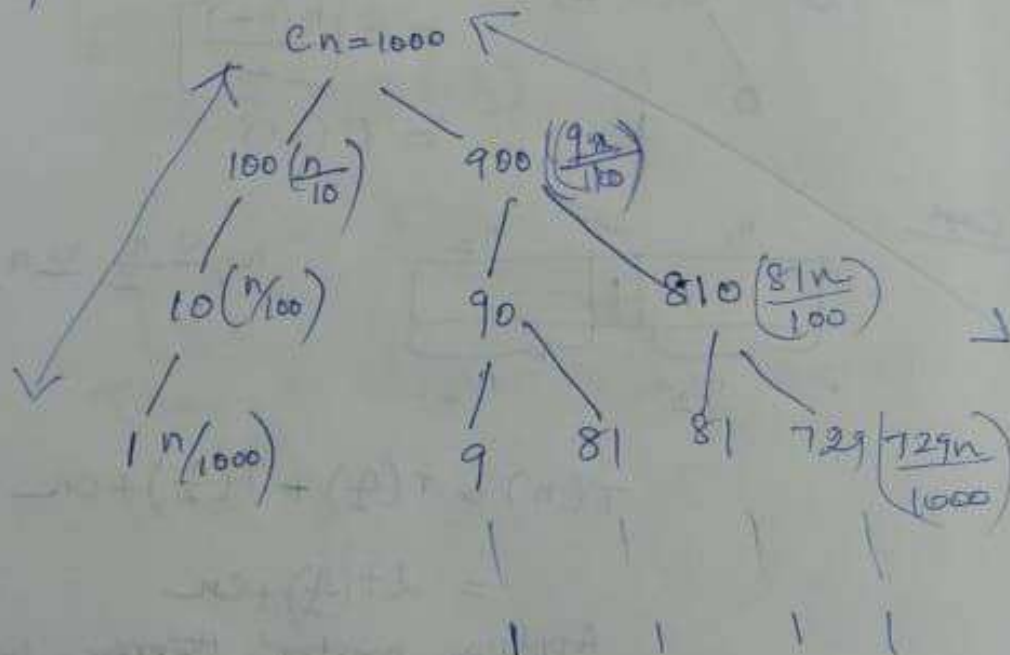
h

$4(2^3)$

$8(2^3)$

$2^{h+1}(1)$

Right sub tree is longer than left subtree because for $n=1000$



$$\sum_{i=0}^R C_n + 2^R T(1)$$

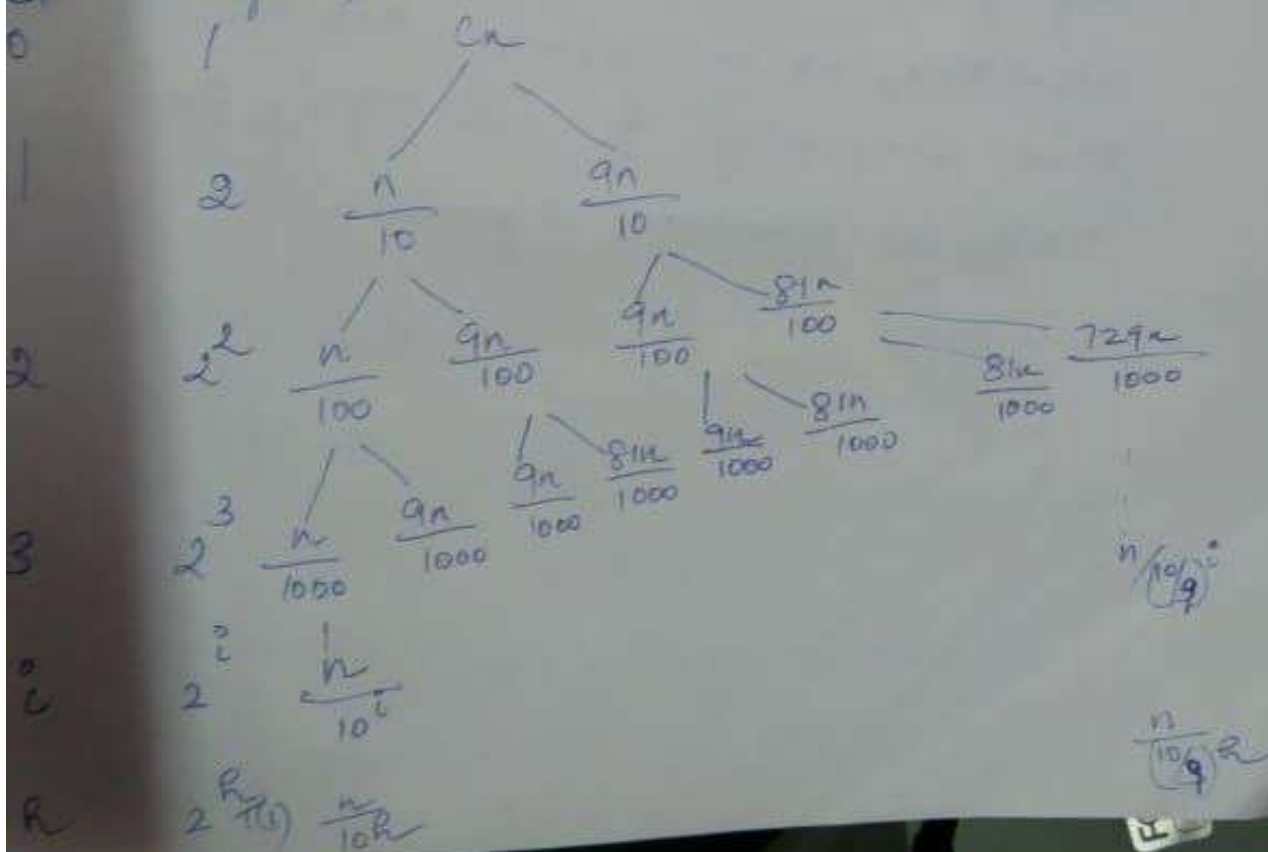
To find 'R'

$$n / (10/9)^R = 1$$

$$n = \left(\frac{10}{9}\right)^k$$

$$T(n) = cn \log n + n^{\log_{10/9} 2} + (1)$$

No of nodes
1



Even for almost best case, the time complexity is $O(n \log n)$

Randomized Quicksort

In the worst case, Quicksort algorithm has a time complexity of $O(n^2)$. In order to fix the problem which occurs when the array is sorted, we go for randomized Quicksort. So in the randomized Quicksort algo we write the full steps ~~where~~ ^{right before the} pivot element is fixed.

```
m = random(l, r) // random fn returns  
Swap(A[l], A[m]) // a random no. in  
                    the range of l to r  
p = A[l]
```

Using probabilistic analysis of algorithms, we can prove that the avg time complexity is $O(n \log n)$ for randomized Quicksort algorithm.